

MASTERING ANSIBLE VAULT

A structural blueprint for securing sensitive infrastructure data.

What is it?



A native, out-of-the-box Ansible CLI tool used to encrypt sensitive data (passwords, API tokens, AWS keys) within playbooks and roles using AES256 encryption.

The Critical Problem

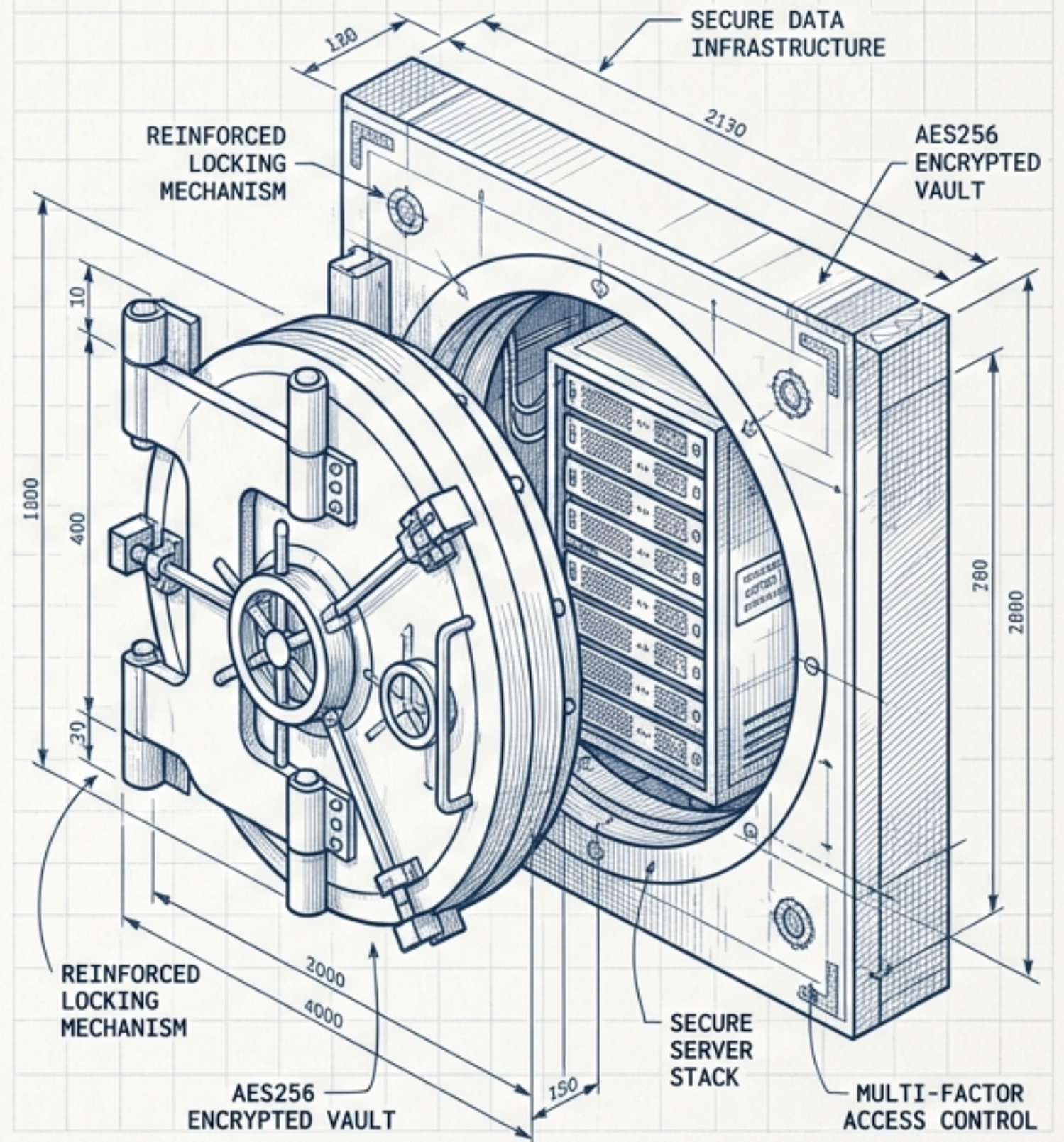


Hardcoding AWS Access and Secret keys in standard YAML playbooks **instantly compromises** your entire infrastructure the moment they are pushed to shared git repositories.

Real-World Use Case



Securing the ``ec2_access_key`` and ``ec2_secret_key`` variables required for automated AWS EC2 instance provisioning via the ``boto3`` module.



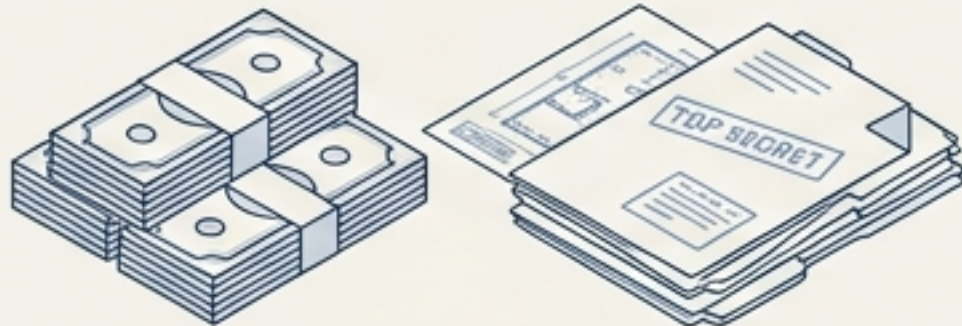
THE SYMMETRICAL ENCRYPTION MENTAL MODEL

Sensitive infrastructure data is dangerously exposed in plain-text YAML files.

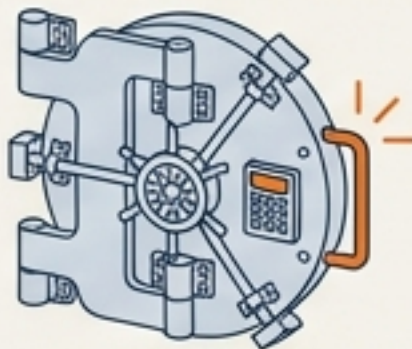
THE PHYSICAL BANK VAULT



Provide an initial password to open the safe.



Place cash or sensitive documents inside.



Close the door; it requires the exact same key to reopen.

THE MASTER KEY

THE DEPOSIT

THE LOCK

THE ANSIBLE VAULT

vault_password.txt

```
6f2a8c4b7d9e1f03a5b2c8e7d9f1a0c4
```

Generate a Vault Password (e.g., via OpenSSL).

playbook.yml

```
---
- hosts: all
  vars:
    ec2_access_key: AKIA...
    db_password: super_secret_password
```

Input standard variables into the playbook file.

playbook.yml (encrypted)

```
$ANSIBLE_VAULT;1.1;AES256
6661323065353336393539396138303735356336306431610a363861386632363332613538643261
376461663566306533363365653065653539623632366264630a3139366466363038396261383836
64393561383533396231626363663730626439313663653537653831646134633166303864373134
```

Save the file; Ansible scrambles it into unreadable ciphertext.

CORE VAULT COMPONENTS



Encrypted Files

Definition: Entire YAML files (e.g., `aws_credentials.yaml`) scrambled via AES encryption.

Use Case: Safely grouping and storing multiple related variables in shared network locations, such as a `group_vars/all` directory.



Vault Password File

Path: `vault.pass`

Definition: A local file containing the master decryption key, typically a secure base64 encoded string generated via OpenSSL.

Use Case: Automates decryption. Prevents the CLI from halting to manually prompt the user for a password during automated playbook runs.



Encrypted Strings

Definition: Encrypting a single variable's value while keeping the rest of the file in readable plain-text.

Clarification: You do not always have to encrypt the whole file. String encryption hides only specific targeted values (like an API token) inside standard playbooks.

COMMAND FUNCTION MATRIX

Rule: All operations must be prefixed with ``ansible-vault``

Command Syntax	Core Function	Trigger Scenario	Resulting State
<code>`create <file>`</code>	Opens editor to create a new file	Starting a fresh playbook	Saves directly as an encrypted file on disk.
<code>`encrypt <file>`</code>	Secures an existing plain-text file	Inherited an unencrypted file with sensitive data	Converts file in-place to encrypted ciphertext.
<code>`decrypt <file>`</code>	Removes encryption permanently	File no longer requires active security	Restores data to standard plain-text YAML.
<code>`edit <file>`</code>	Opens an encrypted file for modification	Adding or removing a variable (e.g., a new API token)	Safely edits without ever exposing plain-text to the disk.
<code>`view <file>`</code>	Prints decrypted content temporarily	Need to quickly check a specific value	Read-only terminal output; file remains fully encrypted.

STEP-BY-STEP PROCEDURE: SECURING AWS CREDENTIALS

```
$ openssl rand -base64 32 > vault.pass
```

Step 1: Generate Cryptographic Password.

Replaces weak passwords with a secure, random 32-byte encoded string.

```
$ cd group_vars/all
```

Step 2: Navigate to Target Scope.

Ensures the encrypted variables are globally accessible to all hosts in the inventory.

```
$ ansible-vault create aws_credentials.yaml  
--vault-password-file vault.pass
```

Step 3: Create the Vault File.

Immediately binds the new file to the secure password file, bypassing manual CLI prompts.

```
ec2_access_key: AKIA...  
ec2_secret_key: 8xP9...  
:wq
```

Step 4: Input Variables & Save.

Inside Vim, input standard YAML. Upon saving, a `cat` command will only reveal \$ANSIBLE_VAULT;1.1;AES256`.

ADVANCED TARGETING & AUTOMATION FLAGS

Targeted String Encryption

Scenario: Hide only the value of the variable `Fubar` inside an otherwise unencrypted file.

```
ansible-vault encrypt_string  
'my_api_token' --name 'Fubar'  
--vault-password-file vault.pass
```

Result: The CLI outputs a specific block of encrypted YAML text that can be copied and pasted directly into any standard, readable playbook.

Critical Automation Flags

Flag: `--vault-password-file <path>`

Mechanism: Tells Ansible exactly where to find the decryption key. Used during execution to prevent the automation pipeline from hanging while waiting for human input.



FATAL MISTAKE: Never commit the `.pass` file to Git. Doing so instantly compromises all associated encrypted files globally.

END-TO-END AUTOMATION WORKFLOW

1. Install Prerequisites

Install **boto3** python module and the AWS Ansible Collection.

2. Secure Credentials

Use **ansible-vault create** for AWS Access/Secret keys.

3. Map Variables

Map Vault keys to playbook variables using Jinja2 templating style.

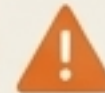
4. Execute Playbook

Run the playbook utilizing the **--vault-password-file** flag.



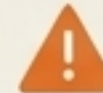
What if skipped?

Ansible cannot authenticate or talk to AWS APIs; playbook execution fails immediately.



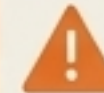
What if skipped?

Credentials sit in plain text. Pushing to GitHub exposes and compromises AWS infrastructure.



What if skipped?

The playbook relies on hardcoded data, rendering the Vault file entirely useless.



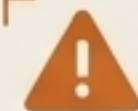
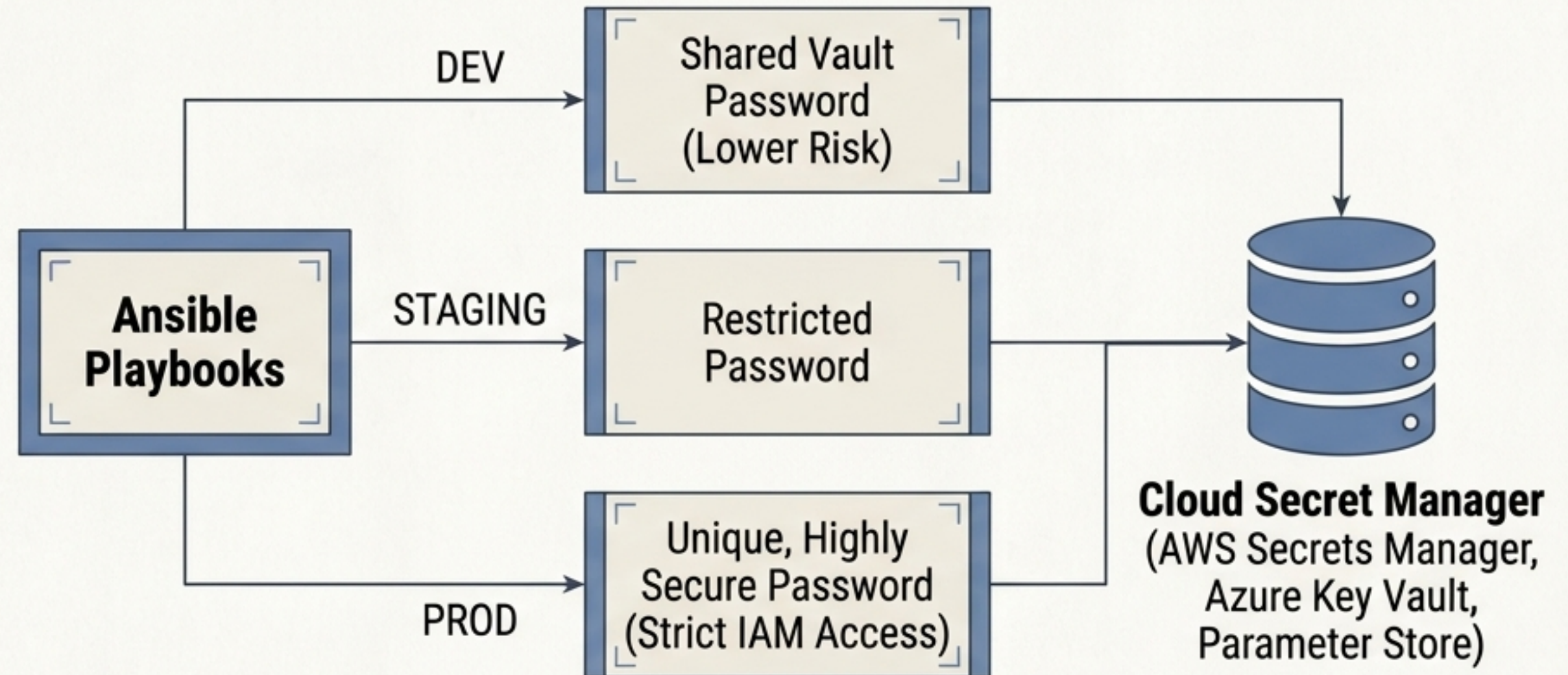
What if skipped?

Playbook automation halts, waiting for a human, or throws a 'Decryption failed' error.

ENTERPRISE BEST PRACTICES: ENVIRONMENT SEGMENTATION

The Scaling Dilemma

- Using ONE password for 100 playbooks is a massive security risk.
- Using 100 different passwords is an unmanageable administrative nightmare.



Secret Management Integration: Never leave `vault.pass` sitting on a random local VM accessible by all developers. Store master passwords in **enterprise Secret Management Services**.

DIAGNOSTIC MATRIX: TROUBLESHOOTING FAILURES

Symptom / Mistake	Root Cause Analysis	Immediate Fix
Encrypted files pushed to git are instantly readable by malicious actors.	The <code>vault.pass</code> file was accidentally committed to the shared repository alongside the YAML files.	Immediately rotate AWS keys, generate a new vault password, and add <code>vault.pass</code> to the <code>.gitignore</code> file.
Playbook stalls indefinitely or fails during an automated CI/CD pipeline run.	Forgot to pass the decryption key location to the automated playbook execution command.	Append <code>--vault-password-file vault.pass</code> to your <code>ansible-playbook</code> execution command.
Playbook executes, but AWS authentication fails with undefined variable errors.	The key name inside the Vault (e.g., <code>ec2_access_key</code>) does not precisely match the variable called in the playbook.	Open the vault using <code>ansible-vault edit</code> and ensure an exact 1:1 string mapping between Vault key and Playbook variable.

2-MINUTE REVISION: CORE PRINCIPLES

The What: Native tool to encrypt sensitive data (AES256).

The Why: Stops critical AWS key leakage in Git.

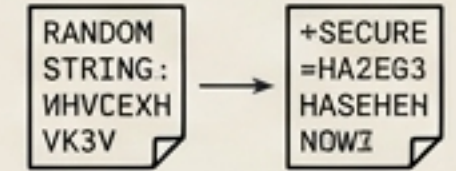
1. Symmetrical Security

Vault uses the exact same password to lock and unlock.



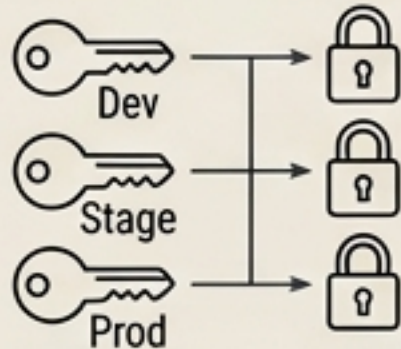
2. Strong Generation

Never use plain-text passwords; use openssl base64 encoded strings.



3. Segment Risk

Always separate master passwords by environment (Dev / Stage / Prod).



4. Cloud Storage

Store master passwords in Cloud Secret Managers, never local VMs.



Command Quick Sync

create (new file) | **encrypt** (existing file) | **edit** (modify secure) |
decrypt (remove secure) | **view** (read-only print) | **encrypt_string** (single variable)

PRACTICE & INTERVIEW PREPARATION

Practice Tasks

Card 1 (Beginner)

Create a `credentials.yaml` file, encrypt it with a basic password, use `view` to read it, and then permanently `decrypt` it.

Practice Tasks

Card 2 (Intermediate)

Generate an `openssl` password, save it to `vault.pass`, and use it to `edit` an existing encrypted file without an interactive prompt.

Practice Tasks

Card 3 (Challenge)

Write an EC2 creation playbook that dynamically pulls an API token from an encrypted string rather than an encrypted file.

Interview Q&A

Card 4 (Basic)

Q: Does Ansible Vault require external plugins?

A: No, it comes natively out-of-the-box with Ansible.

Interview Q&A

Card 5 (Scenario)

Q: You inherited an unencrypted file with live API keys. How do you secure it without rewriting it?

A: Run the `ansible-vault encrypt <file>` command in-place.

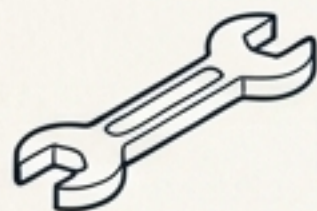
Interview Q&A

Card 6 (Architecture)

Q: How do you manage vault passwords safely at scale?

A: Segment passwords by Dev/Stage/Prod environments and store them in AWS Secrets Manager.

RESOURCE LIBRARY & PREREQUISITES



Core Technologies & Prerequisites

Ansible Vault

Native CLI Tool for AES encryption

AWS Collection

Ansible Galaxy prerequisite

`boto3`

Python module required for AWS API interaction

OpenSSL

Used for cryptographic password generation



Mentioned Secret Managers

AWS Systems Parameter Store

AWS Secrets Manager

Azure Key Vault

(Use these to store `vault.pass` instead of local VMs)



Course Tracking References

Series Source:

Ansible Zero to Hero Series - Episode 9

GitHub Target Folders:

- `Day 2`
(General file scripts)
- `Day 6`
(EC2 instance creation logic)
- `Day 7`
(AWS connection prerequisites)